

1. Microcontroller Fundamental

1. What is the microcontroller?

A microcontroller is a compact integrated circuit to perform a specific function in embedded system.

It contain the

CPU – executes instructions

Memory -Flash/ROM(program), SRAM(Data), EEPROM(non-volatile)

Peripherals – Timers, ADC, UART, I2C, SPI, GPIO etc

2. Difference between mcu and mpu

feature	Microcontroller	Microprocessor
Components	CPU+RAM+ROM+IO on a single chip	Only CPU (external memory and IO required)
Usage	Specific Control Task	General purpose
Cost & Power	Low cost, Low power	High cost, high Power
ex	8051, PIC, STM32, ESP32	Intel i5, AMD Ryzen

3. What are the key components inside a microcontroller?

CPU/Core:- Executes the instructions

Program Memory(Flash) – Stores Firmware

Data Memory (SRAM) – Temporary Data Storage

EEPROM/NVRAM – Non Volatile user data

GPIO Ports – Interface to the external devices

Timer/Counters – Measure time or count the events

Communication Interfaces – UART, SPI, I2C, CAN etc

Interrupt controller:- Manages the interrupts

Clock Source:- Crystal or Internal RC oscillator

4. What are the key components inside a microcontroller?

GPIO – general purpose Input output pins are configurable digital pins that can act as either input or output

5. What is the difference between Harvard and Von Neumann architecture ?

Feature	Harvard	Von Neumann
Memory	Separate for program and data	Common memory for both
Bus	Two separate buses	Single bus
Speed	Fast	Slow
Used in	Most modern MCU- ARM, PIC, AVR	Early CPUs

6. What is the function of the Timer/counter in microcontroller?

Timer:- Generate precise time delays or periodic interrupt

Ex. Generate the 1ms tick for system timing

Counter:- Count external events (pulses, encoder signals)

Ex. Event capture, PWM generation

7. What is the difference between Flash, EEPROM and RAM memory?

Microcontroller memory stores the program code and data that the Central Processing Unit (CPU) needs to perform its functions. It is crucial for the operation of the device and typically combines different types of memory on a single chip, unlike larger computers that often use external memory.

Types of microcontroller memory

Microcontroller memory is primarily categorized based on its function and whether it is volatile or non-volatile. Volatile memory loses its contents when power is removed, while non-volatile memory retains its data.

Volatile memory

- **Random-Access Memory (RAM):** Used for temporary data storage during program execution.
 - **Static RAM (SRAM):** Faster and holds data as long as power is on, but it is more expensive than DRAM. Microcontrollers often use SRAM for variables, the program stack, and the heap.
 - **Dynamic RAM (DRAM):** Cheaper than SRAM but requires a periodic refresh to retain data. It is less common for on-chip microcontroller memory.
- **CPU Registers:** A very small amount of high-speed memory within the CPU that holds data like operands, memory addresses, and the runtime state of the program.

Non-volatile memory

- **Flash Memory:** The most common type of non-volatile memory for storing a microcontroller's program code (firmware).
 - It can be electrically erased and reprogrammed many times, making it easy to update the firmware.

- Flash memory is generally larger than RAM on a microcontroller because the program code is typically larger than the data it needs to process.
- **Electrically Erasable Programmable Read-Only Memory (EEPROM):** A non-volatile memory used to store smaller amounts of data that must persist when the power is off, such as user settings or calibration data.
- **Read-Only Memory (ROM):** Stores program code that is hard-coded during the manufacturing process and cannot be changed. It is a cheaper alternative for high-volume products where the code will not need updates.
- **One-Time Programmable (OTP) ROM:** A type of ROM that can be programmed once by the user and then becomes read-only.

How microcontroller memory works

The microcontroller's CPU executes instructions by working closely with its memory components. This process can be understood by following these steps:

1. **Boot-up:** When the microcontroller is powered on, the CPU retrieves the first instruction from a specific address in the program memory (flash or ROM).
2. **Fetch and Decode:** The CPU fetches an instruction from memory and decodes it to determine the operation that needs to be performed.
3. **Execute:** The CPU executes the instruction. This often involves one of the following tasks:
 1. Reading or writing data from or to the volatile data memory (RAM).
 2. Performing arithmetic or logic operations on data.
 3. Interacting with input/output (I/O) peripherals, such as timers or analog-to-digital converters (ADCs), to receive data from sensors or control external components.
4. **Repeat:** The cycle of fetching, decoding, and executing instructions continues based on the program stored in non-volatile memory.
5. **Data handling:**
 1. **Temporary data:** Variables and intermediate results are stored in the volatile RAM. When a function is called, the microcontroller allocates space on the stack within the RAM for local variables. When the function returns, this memory is freed.
 2. **Persistent data:** Important settings or logged data that must survive a power-off event are written to non-volatile memory like EEPROM or flash.

8. What is the difference between Flash, EEPROM and RAM memory?

A **Watchdog Timer (WDT)** is a hardware timer that resets the microcontroller if the software gets stuck or fails to reset the WDT periodically.

💡 It ensures **system reliability** — prevents hang or infinite loop conditions.

9. What is the purpose of interrupts in microcontrollers?

An **interrupt** allows the MCU to pause normal execution and jump to a predefined ISR (Interrupt Service Routine) when an event occurs (like a button press, data received, timer overflow).

Used for real-time response without constant polling.

10. What is polling? How is it different from interrupt-based control?

Feature	Polling	Interrupt
Definition	MCU continuously checks a flag or status	MCU responds automatically to an event
CPU uses	High	Efficient
Response time	slower	Faster
Example	Checking UART flag in loop	UART Rx interrupt triggers on data arrival

11. What is the difference between volatile and const in Embedded C?

Const:- Value cannot change the during program execution

Volatile:- volatile tell the compiler to do not optimise the variable as it may change at any time

Ex.

Volatile uint8_t flag; // may change inside ISR

Const uint8_t id = 5; // fixed value

12. What is the difference between static and global variables?

Property	static (global scope)	global
Scope	Limited to the file in which it is declared	Accessible from any file (via extern)
Lifetime	Entire program	Entire program
Use case	Hide internal module data	Share data between modules

13. What is the difference between a function call and ISR execution?

Aspect	Normal Function	ISR
Call	Explicit by code	Triggered by hardware/event
Return	Returns normally	Uses special return (reti/iret)
Interrupts	Usually enabled	Usually disabled within ISR
Timing	Predictable	Asynchronous

To handle a shared variable between ISR and main program safely declare it volatile.

14. What is memory mapped registers?

In MCUs, **peripherals are accessed as memory addresses** instead of using I/O instructions.

Each register (GPIO, ADC, Timer, etc.) resides at a fixed memory location.

```
#define GPIOA_ODR (*(volatile uint32_t*)0x48000014)
```

```
GPIOA_ODR |= (1 << 5); // set PA5 high
```

15. Explain UART

UART(universal Asynchronous Receiver transmitter) enables serial communication between devices.

Data format: start bit + data bits (5–9) + optional parity + stop bit(s).

No clock line → asynchronous.

Example: 115200 baud, 8N1 (8 data, no parity, 1 stop).

16. Difference between synchronous and asynchronous communication.

Type	Clock Cycle	Example	Advantages
Synchronous	Shared clock between master & slave	SPI, I2C	Faster, Reliable timing
Asynchronous	No clock, timing via baud rate	UART	Fewer wires, simpler

17. How I2C Protocol works?

Two lines: SDA (data), SCL (clock)

Master–Slave architecture

Each device has a **7/10-bit address**

Master sends **START → Address + R/W bit → ACK → Data → STOP**

Supports multiple masters/slaves

Open-drain pins with pull-ups

18. Explain SPI protocol and how it differs from I²C.

Four lines: MOSI, MISO, SCK, CS (Chip Select)

Full-duplex communication

Faster than I²C (tens of MHz)

No acknowledgment mechanism

Each slave has its own **CS** line

19. What is baud rate?

Number of bits transmitted per second in serial communication.

Example: 9600 baud = 9600 bits/s.

Sender & receiver must use the **same baud rate** for correct timing.

20. What is DMA, and why is it used?

DMA (Direct Memory Access) transfers data between peripherals and memory without CPU involvement.

☐ Frees CPU for other tasks

☐ Faster data transfer

☐ Reduced power consumption

Used in ADC sampling, UART RX/TX, SD card operations, etc.

21. What is a timer? How is it configured?

A **timer** counts clock pulses from a prescaled system clock.

You can configure:

Clock source (internal/external)

Prescaler (divider)

Auto-reload value (period)

Interrupt on overflow

Used for time delays, PWM, frequency measurement.

22. How can you generate a 1 kHz square wave using a timer?

Assume MCU clock is 16MHz

Select prescaler = 16 -> timer tick = 1 microsec

Period = 1ms (1KHz -> 1000Hz)

Set auto reload = 1000 - 1 = 999

Toggle pin on timer interrupt -> Square wave output at 1 KHz

23. What is the PWM

A technique where the **width of the ON pulse** in a fixed-frequency signal is varied.

Used for speed control, LED dimming, voltage regulation.

Duty Cycle = (Ton / Ttotal) × 100 %

24. Explain How PWM can control motor speed or LED brightness

For motors: Average voltage $\propto$ Duty Cycle $\rightarrow$ higher duty = higher speed.

For LEDs: Human eye averages brightness $\propto$ Duty Cycle.

Example: 25 % duty = dim; 75 % = bright.

25. What is ADC (Analog-to-Digital Converter)? How does it work?

ADC converts an **analog voltage** (0–Vref) into a **digital number**.

Uses sampling & quantization.

Resolution = number of bits (e.g., 10-bit $\rightarrow$ 1024 levels).

Conversion time depends on clock & sampling rate.

Example: for 10-bit ADC and Vref = 5 V,

1 LSB = 5 V / 1023 $\approx$ 4.89 mV.

26. How do you handle low-power modes in microcontrollers?

Use MCU sleep/stop modes: reduce CPU clock or shut down peripherals.

Disable unused peripherals (ADC, UART, etc.).

Lower clock frequency when full performance isn't needed.

Wake-up sources: external interrupt, RTC alarm, timer, or communication event.

Example:

```
esp_light_sleep_start(); // ESP32 light sleep
```

Goal: minimize current while maintaining required responsiveness.

27. What is debouncing? How do you implement it in code?

Mechanical switches "bounce," producing rapid on/off transitions.

Software debouncing:

Wait a few milliseconds after a change before accepting it.

```
if (buttonPressed) {  
    _delay_ms(20);  
    if (buttonStillPressed) handlePress();  
}
```

Hardware debouncing:

RC filter or Schmitt trigger.

28. What is RTOS? How is it different from bare-metal programming?

Feature	Bare-metal	RTOS
Task Handling	One loop, manual scheduling	Multitasking with scheduler
Timing	Cooperative	Deterministic
Communication	Global vars/flags	Queues, semaphores
Example	Arduino loop()	FreeRTOS tasks

RTOS adds predictability, modularity, and real-time response.

29. Explain the task scheduling in FreeRTOS?

Pre-emptive scheduler: highest-priority ready task runs.

Tick interrupt: occurs every few ms; context switch may happen.

States: Running, Ready, Blocked, Suspended.

APIs:

- xTaskCreate() – create task
- vTaskDelay() – block for delay
- xQueueSend() / xQueueReceive() – task communication

30. What is ISR latency? How can it be minimized?

ISR latency = time between interrupt request and actual ISR execution.

Minimize by:

Keeping ISRs short

Using high-priority interrupts

Disabling interrupts only when necessary

Avoiding long critical sections

31. How would you design firmware to update over-the-air?

Partition flash:

- Slot A (running firmware)
- Slot B (new image)
- Bootloader region
-

Download image via Wi-Fi/GSM.

Verify checksum/signature.

Set boot flag to new image.

Reboot → bootloader swaps to Slot B.

If failure → rollback to Slot A.

Used in ESP32, STM32 (dual-bank), etc.

32. Explain how bootloaders work.

Bootloader = small program that runs first after reset.

Responsibilities:

Initialize clocks & peripherals minimally

Check for update request (UART, USB, GSM, etc.)

Program new firmware to Flash

Jump to main application start address

Bootloaders ensure safe firmware upgrade without external programmer.

33. How do you debug a microcontroller-based system?

Software methods: UART logs, LED toggling, printf-style tracing.

Hardware tools: JTAG, SWD, logic analyzer, oscilloscope.

IDE tools: breakpoints, watch variables, call stack.

Systematic approach: isolate layers — hardware → drivers → app.

34. How to detect stack overflow in embedded software?

Enable **stack overflow hook** (FreeRTOS: `vApplicationStackOverflowHook()`).

Place **stack guard pattern** (0xAA) and monitor if overwritten.

Use **watchdog** or MPU protection (if supported).

35. How would you identify if a microcontroller is stuck in a loop or crash state?

Use **watchdog timer** reset.

Toggle a “heartbeat” LED periodically; if stopped → system hang.

UART debug prints to trace progress.

Use JTAG to inspect PC (Program Counter).

36. What is a “hard fault”? How do you handle it?

Occurs in ARM MCUs when invalid memory access or divide-by-zero happens.

Handling:

Implement `HardFault_Handler()` to log stack frame, LR, PC.

Identify offending instruction using debugger.

Protect pointers and stack integrity.

37. How do you use breakpoints and watch variables in debugging?

Breakpoint: pause execution at specific code line to inspect state.

Conditional breakpoints: trigger only on specific conditions.

Watch variable: continuously monitor variable changes in real-time.

Useful for verifying logic flow and variable updates.

38. How to interface a sensor with a microcontroller?

Identify interface type (Analog, I²C, SPI, UART).

Configure pins & peripherals.

Calibrate ADC or communication speed.

Apply filtering or scaling to raw data.

Implement periodic sampling via timer/RTOS task.

39. How would you interface an LCD or OLED display?

For **character LCD (HD44780)**: use 4-bit or 8-bit parallel mode.

For **OLED (SSD1306)**: use I²C/SPI.

Initialize per datasheet, send commands & data bytes, control cursor/graphics.

40. How to interface a motor driver using PWM signals?

Connect MCU PWM pin to driver input (e.g., L298N, DRV8833).

Vary PWM duty cycle to control speed.

Use two pins for direction (IN1, IN2).

Optionally add feedback (encoder) for closed-loop control.

41. How do you handle communication between two microcontrollers?

Choose protocol: UART (simple), SPI (fast), I²C (multi-master), CAN (robust).

Define data frame format (header, payload, checksum).

Implement acknowledgment and retry mechanism.

Ensure voltage-level compatibility.

42. How would you implement firmware update via GSM/Wi-Fi/Bluetooth?

Use GSM/Wi-Fi to download binary file (HTTP/MQTT).

Store in secondary flash area.

Verify checksum/signature.

Use bootloader to replace old firmware after reboot.

(Same principle as OTA; only transport layer differs.)

43. How do you store and retrieve configuration data (EEPROM/SPIFFS)?

EEPROM: byte-wise read/write, use wear-leveling.

SPIFFS/LittleFS (ESP32): file-based storage on Flash.

Store parameters as key-value pairs, JSON, or simple text.

Retrieve at boot and apply to system variables.

44. What are the common causes of MCU resets and how to diagnose them?

Cause	Description	Detection
Watchdog Reset	Software hang	WDT flag
Brown-out	Low supply voltage	BOR flag
Power-on Reset	On startup	POR flag
External Reset	Reset pin pressed	EXTR flag
Software Reset	By code (NVIC_SystemReset())	SWR flag

45. Explain UART communication.

UART (**Universal Asynchronous Receiver Transmitter**) is a serial communication protocol that transfers data **bit by bit** between two devices **asynchronously** (no common clock signal).

Key Features:

- Uses **TX** (transmit) and **RX** (receive) lines.
- Data format: Start bit → Data bits (usually 8) → Optional parity bit → Stop bit(s).
- Baud rate defines speed (e.g., 9600, 115200 bps).

Example:

Microcontroller ↔ GSM module (AT commands over UART).

46. Difference between synchronous and asynchronous communication.

Type	Clock Signal	Example	Description
Synchronous	Yes	SPI, I ² C	Data sent with a clock — devices stay in sync.
Asynchronous	No	UART	Devices agree on baud rate; each frame has start/stop bits for timing.

47. How does I²C protocol work?

I²C (**Inter-Integrated Circuit**) is a **two-wire** serial protocol used for communication between master and multiple slave devices.

Lines: SDA (Data), SCL (Clock)

Addressing: Each slave has a unique 7-bit or 10-bit address.

Master generates: Start condition, clock, and stop condition.

Data transfer: Byte by byte, with ACK/NACK after each byte.

Example:

Microcontroller (Master) ↔ Temperature sensor (Slave).

48. Explain SPI protocol and how it differs from I²C.

SPI (Serial Peripheral Interface) is a **high-speed synchronous** communication protocol.

Lines:

- **MOSI** (Master Out Slave In)
- **MISO** (Master In Slave Out)
- **SCK** (Clock)
- **CS/SS** (Chip Select)
- **Full duplex** communication.
- Faster than I²C but requires more pins.

Feature	SPI	I ² C
Speed	Faster	Slower
Wires	4	2
Communication	Full duplex	Half duplex
Addressing	Via CS pin	Device address
Hardware complexity	High	Low

49. What is the role of chip select (CS) in SPI?

The **CS (Chip Select)** or **SS (Slave Select)** pin is used by the **master** to select which slave device it wants to communicate with.

When **CS = LOW**, the slave is active.

When **CS = HIGH**, the slave is inactive (ignores bus signals).

50. What is baud rate?

Baud rate = Number of bits transmitted per second.

For example, **9600 baud** means 9600 bits per second.

Note:

Baud rate must match between transmitter and receiver for correct communication.

51. What happens if two UART devices have mismatched baud rates?

Receiver samples data at wrong intervals.

Frame misalignment occurs.

Characters are received incorrectly or lost.

Communication becomes **garbled or unreliable**.

52. What is DMA, and why is it used?

DMA (Direct Memory Access) allows peripherals to access system memory **without CPU intervention**.

Improves speed and efficiency.

CPU can perform other tasks while DMA moves data.

Used in **ADC → Memory, UART → Buffer**, etc.

Example:

ADC samples continuously via DMA → data stored in buffer → CPU reads later.

53. What is a timer? How is it configured?

A **timer** is a hardware peripheral that counts clock pulses to measure time intervals or generate precise delays.

Configuration steps:

Select clock source (internal/external).

Set prescaler (to slow down clock).

Set auto-reload or compare value.

Enable interrupt or output mode.

Use cases: delays, PWM generation, frequency counting.

54. Difference between timer and counter mode.

Mode	Input Source	Purpose
Timer	Internal clock	Measures time duration.
Counter	External pulses	Counts external events or signals.

Example:

Timer: Generate 1-second delay.

Counter: Count number of pulses from a sensor.

55. How can you generate a 1 kHz square wave using a timer?

Configure timer in toggle/output compare mode.

Set timer frequency = $\text{MCU clock} / (\text{prescaler} \times \text{period})$.

For 1 kHz wave → Period = 1 ms.

Example (STM32):

If MCU = 16 MHz,

Prescaler = 16,000 → Timer counts every 1 ms.

Toggle pin → 1 kHz square wave output.

56. What is PWM (Pulse Width Modulation)?

PWM controls **average power** delivered to a load by rapidly switching a signal **ON** and **OFF**.

$$\text{Duty Cycle (\%)} = (\text{TON} / (\text{TON} + \text{TOFF})) \times 100$$

0% → Always OFF

100% → Always ON

57. What is ADC? How does it work?

ADC (Analog to Digital Converter) converts an analog voltage signal into a digital value.

Steps:

Sampling the analog input.

Quantizing it to discrete levels.

Encoding to binary representation.

Example:

For 10-bit ADC (0–1023) and reference 5V:

Analog 2.5V → Digital = $(2.5/5) \times 1023 \approx 512$.

58. What is the difference between 8-bit and 10-bit ADC?

Parameter	8-bit ADC	10-bit ADC
Resolution	$2^8 = 256$ levels	$2^{10} = 1024$ levels
Step Size (5V ref)	19.53 mV	4.88 mV
Accuracy	Lower	Higher

So, a **10-bit ADC** gives finer measurement accuracy of analog signals.

(UART, I²C, SPI, CAN), how it works, strengths, weaknesses, and typical use-cases.

UART (Universal Asynchronous Receiver/Transmitter)

What it is / how it works

- Asynchronous serial point-to-point link (no shared clock). Data framed into start bit, data bits, optional parity, stop bits. Sender & receiver must agree on baud rate.
- Physical layer on microcontrollers is TTL-level UART. For longer distances or industrial use you use transceivers (RS-232, RS-485).

Strengths

- Simple and ubiquitous — trivial to implement and debug.
- Works with very little hardware (2 wires TX/RX, optionally GND).
- Good for human-readable logs, simple device-to-device comms, bootloaders, modems, GPS modules, BLE serial, sensors with UART.
- Can be extended to multi-drop via RS-485 transceivers (differential) for longer distances and multi-node.

Weaknesses

- Point-to-point by default (no addressing), not multi-master.
- No built-in device discovery or bus arbitration.
- Flow control must be added (RTS/CTS or XON/XOFF) if needed.
- Timing sensitive — both sides must match baud rate.

Speeds & distances

- Common: 9.6k, 115.2k bps. MCU UARTs can often do 1–3 Mbps; some support multi-Mbps.
- Over RS-485 you can reach hundreds of kbps over tens to hundreds of meters (depends on cable & speed).

Typical use-cases

- PC ↔ MCU console, debug logs, GPS, Bluetooth modules, serial sensors, simple device links.
- Industrial multi-drop when using RS-485.

I²C (Inter-Integrated Circuit)

What it is / how it works

- Two-wire serial bus (SDA = data, SCL = clock), multi-master/multi-slave. Uses open-drain outputs and pull-up resistors; master generates clock.
- Devices have 7-bit or 10-bit addresses. Master initiates transactions.

Strengths

- Very simple wiring (2 wires) and supports many devices on same bus.
- Well-suited for short on-board peripheral communication (sensors, EEPROMs, RTCs, ADC/DAC chips).
- Supports clock stretching (slave can hold SCL low to slow master).

Weaknesses

- Open-drain + pull-ups limit high speed on long wires — not ideal for long distances.
- Not full-duplex; transactions are request/response style.
- Bus can become a single point of failure: a stuck SDA/SCL (short or misbehaving device) can hang bus.
- Arbitration and multi-master exist but are rare in simple designs and add complexity.

Speeds

- Standard: 100 kHz (Standard-mode)
- Fast: 400 kHz (Fast-mode)
- Fast-Plus: 1 MHz
- High-Speed: 3.4 MHz (less common)
- Effective bus length: typically <0.5–1 m for higher speeds; can be longer at lower speeds with careful design.

Typical use-cases

- On-board peripherals: temp sensors, EEPROM, accelerometers, small displays, RTCs, power chips, chip configuration registers.

SPI (Serial Peripheral Interface)

What it is / how it works

- Full-duplex synchronous serial link with separate clock (SCLK), master out slave in (MOSI), master in slave out (MISO), and one chip-select (CS) per slave. No formal standard for addressing — CS lines used to select slaves.
- Simple protocol; many variants (CPOL/CPHA settings).

Strengths

- Very fast, low-latency, full duplex; ideal for high-bandwidth on-board links (e.g., high-speed ADCs, displays, SD cards, flash).
- Simple to implement in hardware and software.
- Deterministic timing and very low latency.

Weaknesses

- Requires one CS pin per slave (or additional hardware decoder) — wiring/IO cost can grow.
- Not multi-master by default; no built-in acknowledgement/flow-control.
- No standardised device addressing and no built-in error detection.
- Limited bus length — primarily for on-board short traces (a few tens of cm to a meter in special cases).

Speeds

- Typical tens of MHz (8–50+ MHz on many MCUs); some peripherals support hundreds of MHz.
- Higher speeds demand careful PCB layout and short lines.

Typical use-cases

- Sensors that need high sample rate, displays (TFT), external flash memory, SD cards, audio codecs, FPGA configuration, high-speed ADCs/DACs.

CAN (Controller Area Network)

What it is / how it works

- Multi-master, message-oriented bus with priority-based arbitration. Uses differential twisted-pair physical layer (CAN_H, CAN_L).
- Messages use identifiers (IDs) not addresses; all nodes hear messages and decide if relevant.
- Robust: includes CRC, ACK, retransmission, error counters and automatic fault confinement.

Strengths

- Extremely robust and fault-tolerant, designed for noisy environments (automotive, industrial).
- Multi-master with non-destructive arbitration — high-priority messages pass without collisions.
- Supports many nodes on same bus with deterministic behavior for priority messages.
- Good for medium distances and harsh electrical environments.

Weaknesses

- Protocol complexity — needs CAN controller + transceiver.
- Lower raw payload throughput vs SPI (overhead for frames). Classical CAN up to 1 Mbps; CAN FD higher data phase rates (e.g., up to 8 Mbps in many implementations).
- Not suited for extremely high data-rate streaming (e.g., raw audio/video).
- Message-based (not byte-stream); might need layer/profiles for message meaning.

Speeds & distances

- Classical CAN: up to 1 Mbps (higher speeds reduce cable length).
- CAN FD: data phase up to several Mbps (implementation-dependent, commonly 2–8 Mbps).
- Bus length: up to kilometers at very low speeds; at 1 Mbps typically up to ~40 meters (depends on cable & transceivers).

Typical use-cases

- Automotive communications (ECUs), industrial automation networks, robotics, fieldbus replacements, vehicle sensors, safety systems.

Compact comparison table

Feature / Bus	UART (TTL / RS-232 / RS-485)	I ² C	SPI	CAN
Wires (min)	2 (TX/RX) + GND (RTS/CTS optional)	2 (SDA, SCL) + GND	3 (MISO/MOSI/SCLK) + CS per slave + GND	2 (CAN_H, CAN_L) + GND
Topology	Point-to-point (multi-drop via RS-485)	Multi-master/multi-slave bus	Master-slave (single master typical)	Multi-master message bus
Typical speed	9.6 kbps → multi-Mbps (MCU dependent)	100 kHz – 3.4 MHz (typical: 100k/400k/1M)	MHz — tens of MHz (very fast)	125 kbps – 1 Mbps (Classical); CAN FD higher
Full-duplex?	Yes (point-to-point)	No	Yes	No (message-based)
Addressing	None (use protocol)	7/10-bit addresses	Chip-select lines	Message IDs (priority)
Arbitration / collision	No (RS-485 needs protocol)	Multi-master arbitration exists	Not built-in	Non-destructive arbitration (built-in)
Error detection & recovery	Optional (parity, checksums)	ACK + NACK via bus behaviour; no CRC in I2C frames	None (application layer)	CRC, ACK, retransmission, error counters
Robustness to noise	TTL: low; RS-485: high	Moderate (short traces)	Low (short PCB traces)	High (differential twisted pair)
Typical distance	Short (TTL) → long with RS-485	Short (on-board)	Very short (on-board)	Medium to long (twisted pair)
Device count practical	2 (more with RS-485 multi-drop)	Many (address space)	Many — but CS pins scale	Many
Hardware complexity	Low	Low	Low–Moderate (CS wiring)	Moderate–High (controller + transceiver)
Best for	Simple, human/PC comms, debug, RS-485 networks	On-board low-pin sensors/peripherals	High-speed on-board peripherals	Vehicle/industrial control, robust multi-node networks